

Rapport de Projet : Moteur de Recherche

<https://github.com/LeoAugustinAmy/SearchEngine.git>

AMY Léo

Table des matières

I. Spécifications.....	2
II. Analyse.....	2
II.1 Environnement de travail.....	2
II.2 Données identifiées.....	2
II.3 Diagramme de classes simplifié.....	3
III. Conception.....	3
III.1 Organisation du travail.....	3
III.2 Algorithmes spécifiques.....	3
III.3 Problèmes rencontrés et solutions.....	4
III.4 Exemple d'utilisation.....	4
IV. Validation.....	4
V. Maintenance.....	5

I. Spécifications

L'objectif de ce projet est de réaliser une application Python complète couvrant l'intégralité du cycle de vie d'un logiciel, de la définition du besoin à la maintenance. Le but est de développer un moteur de recherche capable de collecter, traiter et interroger un corpus de documents issus de sources variées, telles que Reddit et Arxiv. Les spécifications, établies à partir des énoncés des travaux dirigés (TD 3 à 10), imposent que le programme puisse récupérer des données via des API, les nettoyer et les stocker de manière structurée.

Le cœur de l'application doit reposer sur un moteur de recherche vectoriel utilisant la pondération TF-IDF et la mesure de similarité cosinus pour classer les documents par pertinence par rapport à une requête utilisateur. Enfin, le projet doit inclure une interface graphique interactive, réalisée au sein d'un notebook, permettant à l'utilisateur de lancer des recherches et de visualiser les résultats accompagnés d'un contexte textuel. Le livrable final doit respecter des contraintes strictes, tel qu'un code modulaire et documenté.

II. Analyse

II.1 Environnement de travail

Pour répondre aux exigences techniques, l'environnement de développement retenu est Python 3.10. Le choix de cet environnement s'explique par sa richesse en bibliothèques dédiées au traitement de données et sa portabilité multi-plateforme. L'interface utilisateur a été conçue sous Jupyter Notebook, en exploitant la librairie ipywidgets pour offrir une interaction dynamique sans nécessiter le développement d'une interface graphique lourde. Pour la manipulation des données, nous avons privilégié pandas pour sa capacité à gérer des structures tabulaires et scipy pour le calcul matriciel, notamment pour la gestion des matrices creuses indispensables au traitement de grands volumes de texte.

II.2 Données identifiées

L'analyse des spécifications a permis d'identifier les structures de données essentielles au fonctionnement de l'application. Le système manipule principalement deux entités : les auteurs et les documents. Les auteurs sont définis par leur nom et l'ensemble de leurs productions. Les documents, quant à eux, partagent des attributs communs tels que le titre, la date, l'auteur et le contenu textuel. Cependant, selon leur provenance, ils possèdent des spécificités propres que le système doit gérer : les documents issus de

Reddit intègrent un nombre de commentaires, tandis que ceux provenant d'Arxiv comportent un résumé et une liste de co-auteurs.

II.3 Diagramme de classes simplifié

La conception repose sur une architecture orientée objet. La classe centrale, nommée Corpus, agit comme le gestionnaire principal des données ; elle contient le dictionnaire des documents et des auteurs, et fournit des méthodes pour le nettoyage de texte et les statistiques globales. La modélisation des documents utilise l'héritage pour gérer la diversité des sources. Une classe mère Document définit la structure de base, tandis que les classes filles RedditDocument et ArxivDocument héritent de ces propriétés tout en implémentant leurs attributs spécifiques. Enfin, une classe SearchEngine est dédiée à la logique de recherche ; elle utilise les données de la classe Corpus pour construire les matrices TF-IDF et effectuer les calculs de similarité, séparant ainsi le stockage des données de leur traitement algorithmique.

III. Conception

III.1 Organisation du travail

Ce projet ayant été réalisé de manière individuelle, la question de la répartition des tâches entre plusieurs collaborateurs ne se pose pas. L'organisation du travail a suivi une progression linéaire, correspondant à l'évolution des travaux dirigés (TD 3 à 10). La première phase s'est concentrée sur la mise en place de la structure des classes et la collecte des données via les API. Dans un second temps, l'effort a porté sur le développement du moteur mathématique (matrice TF-IDF et similarité). Enfin, la dernière phase a été consacrée à l'intégration de l'interface utilisateur et à l'optimisation du code.

III.2 Algorithmes spécifiques

L'implémentation repose sur des algorithmes de recherche d'information adaptés aux contraintes de mémoire. Pour l'indexation, nous avons utilisé la méthode TF-IDF (Term Frequency-Inverse Document Frequency) implémentée via des matrices creuses . Ce choix technique permet d'éviter la saturation de la mémoire vive, car la matrice terme-document contient majoritairement des zéros. La recherche de pertinence s'effectue grâce au calcul de la similarité cosinus entre le vecteur de la requête et ceux des documents. De plus, un algorithme de concordancier a été développé pour extraire les contextes gauche et droit d'un mot-clé en utilisant des expressions régulières, permettant ainsi d'afficher un aperçu à l'utilisateur.

III.3 Problèmes rencontrés et solutions

Plusieurs difficultés techniques ont été rencontrées et résolues au cours du développement. La lenteur et l'instabilité des appels API lors des phases de test constituaient un frein majeur. Pour y remédier, nous avons implémenté des méthodes de sérialisation JSON (`save_json` et `load_json`), permettant de sauvegarder le corpus localement et de travailler sans connexion permanente. Un autre problème concernait la lisibilité des extraits textuels dans l'interface, souvent tronqués par les paramètres par défaut. Ce point a été résolu en configurant les options d'affichage de la librairie Pandas pour garantir la lisibilité des résultats.

III.4 Exemple d'utilisation

Le fonctionnement de l'application suit une séquence logique. L'utilisateur commence par instancier un objet `Corpus` et charge les données, soit via les API, soit depuis une sauvegarde locale. Il initialise ensuite le `SearchEngine`, ce qui déclenche le calcul des matrices. Via l'interface graphique, il saisit un mot-clé (par exemple "Intelligence"). Le programme traite cette requête, calcule les scores de similarité et affiche un tableau présentant les documents les plus pertinents, triés par score décroissant, avec un extrait montrant le mot-clé en contexte.

IV. Validation

La validation du logiciel a été menée de manière méthodique à travers deux types de tests. Des tests simple ont d'abord été réalisés pour vérifier la fiabilité des méthodes critiques de façon isolée. Par exemple, la fonction de nettoyage de texte a été éprouvée avec des chaînes complexes contenant de la ponctuation et des majuscules pour garantir une normalisation correcte. De même, le concordancier a été testé sur des cas particulier pour s'assurer qu'il gérait correctement les débuts et fins de chaînes sans erreur d'indexation.

Des tests globaux ont ensuite été effectués via l'interface utilisateur pour valider le comportement général de l'application. Nous avons simulé des scénarios d'erreur, comme la soumission d'une requête vide ou la recherche d'un terme absent du vocabulaire. Ces tests ont confirmé que l'application gère ces exceptions de manière robuste, en affichant des messages d'information clairs à l'utilisateur plutôt que de provoquer un arrêt du programme.

V. Maintenance

Dans l'optique d'une évolution future du logiciel, plusieurs pistes d'amélioration sont envisageables. Une fonctionnalité intéressante et relativement simple à mettre en œuvre serait l'ajout d'une surbrillance visuelle des termes recherchés directement dans les résultats. Cela nécessiterait uniquement une modification de la méthode d'affichage pour injecter des balises de formatage autour du motif trouvé. En revanche, l'ajout de filtres avancés (par date ou par auteur) représenterait une évolution plus complexe. Cela impliquerait une refonte partielle du moteur de recherche pour permettre de masquer certains documents sur la matrice avant le calcul de similarité, augmentant ainsi la complexité algorithmique du programme.